DORA Reporting Platform

Step-by-Step Build Plan

Phase ordering, task breakdown, and rationale

Document ID: DORA-ARCH-0.4 | Date: 2026-03-13 | Status: Complete

Build Strategy

The simulation engine is built before the dashboard because the dashboard requires populated data to develop and test against. Each phase produces a working artifact before the next phase begins.

Phase	Name	Output
Phase 0	Architecture & Planning	Design docs (this set)
Phase 1	Setup & Config	Two repos, Jira project, config files
Phase 2	GitHub Simulation	6 months of commits, PRs, releases in dora-demo-app
Phase 3	Jira Simulation	6 months of stories and incidents in DORA project
Phase 4	Dashboard Core	Working dashboard reading real API data
Phase 5	Classification & Polish	DORA bands, scorecard, date picker, GitHub Pages deploy

Phase 1 — Setup & Config

#	Task
1.1	Create dora-demo-app GitHub repo (simulation target repo)
1.2	Create dora-platform GitHub repo (dashboard + sim scripts)
1.3	Set up Jira Cloud project (DORA) with issue types and workflow
1.4	Add four custom Jira fields: Deployment Version, First Commit Date, Incident Severity, Linked Release
1.5	Write config.py with API tokens, date ranges (Sep 2025 – Mar 2026), simulation params
1.6	Create .env.example and .gitignore

Phase 2 — GitHub Simulation

#	Task
2.1	Write github_sim.py: backdated commits using GIT_AUTHOR_DATE / GIT_COMMITTER_DATE env vars
2.2	Script PR creation and merge events spread across 6 months (~3–5 PRs/week)
2.3	Script release tag creation after each batch of merged PRs (releases = deployments)
2.4	Inject ~15% failure releases with naming convention vX.Y.Z-hotfix

Phase 3 — Jira Simulation

#	Task
3.1	Write jira_sim.py: create stories linked to feature branches (one story per PR)
3.2	Simulate ticket transitions with backdated timestamps matching GitHub PR timeline
3.3	Create incident tickets for each failure release (within 48h of release date)
3.4	Transition incidents to Resolved after simulated MTTR duration (2–72 hours)
3.5	Run run_simulation.py orchestrator and validate data looks realistic

Phase 4 — Dashboard Core

#	Task
4.1	Build index.html skeleton: header, 4 metric panels, weekly/monthly toggle
4.2	Write js/api.js: fetch wrappers for GitHub /releases, /pulls, /commits; Jira /search
4.3	Write proxy/server.py: local proxy for Jira CORS and Basic Auth injection
4.4	Write js/metrics.js: pure functions computing 4 DORA metrics by week and month
4.5	Write js/charts.js: Chart.js line/bar charts, dataset switching on toggle
4.6	Implement weekly / monthly toggle wired to chart dataset swap

Phase 5 — Classification & Polish

#	Task
5.1	Add DORA performance band overlay lines to each chart (Elite/High/Medium/Low thresholds)
5.2	Add summary scorecard: one badge per metric showing current classification
5.3	Add date range picker to filter chart data
5.4	Style and polish (css/style.css) — clean, professional dashboard look
5.5	Deploy dashboard to GitHub Pages
5.6	Write README.md with setup and run instructions

DORA Performance Band Thresholds

These are the standard DORA Research thresholds used for classification in Phase 5:

Metric	Elite	High	Medium	Low
Deployment Frequency	On-demand (multiple/day)	Weekly	Monthly	< Monthly
Lead Time for Changes	< 1 hour	1 day – 1 week	1 week – 1 month	> 1 month
Change Failure Rate	0–5%	5–10%	10–15%	> 15%

MTTR	< 1 hour	< 1 day	1 day – 1 week	> 1 week